

Technical Interview Assignment

AI Full Stack Developer

Assignment Overview

This assessment is designed to evaluate your practical skills in **frontend development, backend development, AI application development, authentication, business logic, API integrations, and system design**.

You may use AI coding tools such as **Claude Code, OpenAI Codex, Cursor, or similar tools** during the assignment. However, you should be able to explain the implementation, architecture, and technical decisions made during the interview.

Assignment 1 – Figma to Responsive React Page

Objective

Convert the provided **Figma design** into a functional React application with a close visual match.

Figma: <https://www.figma.com/design/Yqq9qC4hZqj0adhv5kJUNG/Untitled?node-id=1-503&t=rz1rvy5igFal5MjK-1>

Requirements

- React + TypeScript
- Tailwind CSS
- Shadcn UI where appropriate
- Closely match the provided Figma design
- Fully responsive across:
 - Desktop/Laptop
 - Tablet
 - Mobile
- Reusable and well-structured components
- Clean and maintainable code
- Good frontend performance

AI Development Tools

You may use:

- Claude Code
- OpenAI Codex
- Cursor
- GitHub Copilot

- Other AI coding assistants

Please mention:

- Which AI tools you used
- How you used them during development
- Total time taken to complete the assignment

Deliverables

- GitHub repository
- Live/deployed URL
- README with setup instructions
- Total development time

Evaluation Criteria

- Figma accuracy
 - Responsive design
 - UI quality
 - Code quality
 - Performance
 - Component structure
 - Development speed
 - Effective use of AI coding tools
-

Assignment 2 – Mini AI E-Commerce Application

Objective

Build a small e-commerce application with **Google authentication, customer and admin access, product/order management, Stripe payment integration, and an AI support agent.**

The application should demonstrate your ability to build a complete flow from:

UI → API → Database → Authentication → Business Logic → AI → Integration

Frontend

Build a simple e-commerce interface with:

- Google Sign-In
- Product listing
- Product details
- Add to cart
- Basic checkout flow
- Customer order history

Frontend Stack

- React
- TypeScript
- Tailwind CSS
- Shadcn UI

Backend

Build the backend using:

- Python
- FastAPI
- MongoDB or PostgreSQL

Implement:

- User authentication
- Customer and Admin roles
- Product APIs
- Order APIs
- Basic RBAC

The backend must enforce authorization. Do not rely only on frontend restrictions.

AI Support Agent

Build a simple AI support agent using **LangChain** or **LangGraph**.

The agent should be able to answer questions such as:

What is the price of Product X?

What products are available?

What is the status of my order?

The AI agent should retrieve actual product and order information through backend APIs/tools rather than relying only on general LLM knowledge.

Required Integrations

1. Google Sign-In Authentication

Implement Google OAuth authentication for customers.

Authentication Flow

Frontend → Google Sign-In → FastAPI → Verify User → Create/Login User → Authenticated Session

The implementation should support:

- Google Sign-In
- New user registration
- Existing user login
- Logout
- Protected customer routes

2. Stripe Payment Integration

Integrate **Stripe in test mode** for the checkout/payment flow.

Payment Flow

Frontend → FastAPI → Stripe Checkout → Payment → Webhook → Payment Verification → Order Status

The implementation should handle:

- Creating a Stripe checkout/payment session
- Successful payment
- Failed/cancelled payment
- Stripe webhook
- Backend-side payment verification
- Updating the order status after successful payment confirmation

Basic Business Logic

The application should correctly handle:

- Product stock
- Cart quantity
- Order creation
- Payment success/failure
- Customer vs Admin permissions
- Customers accessing only their own orders
- Admin-only product/order management
- Preventing unauthorized API access

Simple System Design

Along with the application, provide a **one-page system design** showing:

- Frontend
- FastAPI backend
- Database
- AI Agent
- Google Authentication
- Stripe

- AWS/deployment approach

Also briefly explain:

How would you scale this application if the number of users and AI requests increased significantly?

Deliverables

Please submit:

- GitHub repository
 - Live/demo URL
 - README
 - Database schema
 - Basic API documentation
 - One-page system design
 - Total time taken
 - AI tools used
-

What We Are Looking For

We are looking for a developer who can demonstrate practical ability to build and understand an application end-to-end.

The assessment will focus on:

- Frontend implementation
- Backend/API development
- Authentication
- Business logic
- AI integration
- Third-party API integration
- System design
- Code quality
- Problem-solving
- Development speed
- Effective use of AI development tools

The goal is not to build a very large application. We are evaluating **how you think, how you structure the application, how you solve real-world problems, and how effectively you can build a production-oriented AI application.**